



PROBLEMAS DE ESTRUCTURA DE COMPUTADORES

SEGMENTACIÓN

1. Sea un procesador segmentado con cinco etapas como el visto en clase (DLX) que tiene las siguientes características:

- Se puede escribir y leer en el banco de registros en el mismo ciclo de reloj.
- No tiene técnicas para reducir o eliminar paradas en caso de riesgos de datos (NO tiene implementado cortocircuito).
- Los saltos se resuelven en la etapa ID. Después del salto siempre se busca la instrucción siguiente y en caso de que se produzca el salto se anula la instrucción buscada.

Se ejecuta en dicho procesador una aplicación con las siguientes características:

- El 20 % de las instrucciones son saltos condicionales y el 40 % de los saltos se realizan. No hay dependencias de datos en las instrucciones de salto.
- El 18% de las veces, las instrucciones I_{i+1} tienen una dependencia de LDE con las instrucciones I_i (el 30 % de éstas corresponden a instrucciones de load).
- El 6% de las veces las instrucciones I_{i+2} tienen dependencias de LDE con la instrucción I_i , (el 30 % de éstas corresponden a instrucciones de load), y en esos casos nunca hay dependencias entre I_{i+1} e I_{i+2} .

Calcular:

- a) Los ciclos por instrucción de dicho procesador.
 - b) Los ciclos por instrucción si al procesador se le añade forwarding (cortocircuito).
 - c) SPEED UP del segundo caso frente al primero.
2. Sea un procesador segmentado de cinco etapas con las siguientes características:
- Tiene el HW para la detección de riesgo y generación de la parada en decodificación.
 - Se puede escribir y leer el registro en el mismo ciclo de reloj.
 - Tiene implementada la técnica de forwarding (cortocircuito).
 - Los saltos condicionales se resuelven en decodificación y se solucionan parando el pipeline.

Supongamos que a este procesador se le realiza la siguiente modificación: se unen las etapas de ejecución y memoria. Como consecuencia de esta modificación el ciclo de reloj aumenta un 35%.

Hallar la relación de velocidades entre la versión original del procesador y la modificada cuando se ejecuta una aplicación en la que en el 25% de los casos existen dependencias LDE entre un load y la instrucción que le sigue (esta instrucción nunca es un salto condicional) y que el 33% son saltos condicionales.

3. El siguiente fragmento de código se ejecuta en un DLX segmentado de cinco etapas:

```
sub  R1,R2,R3
add  R4,R5,R6
sub  R5,R4,R8
add  R7,R2,R3
add  R9,R7,R3
lw   R1,10(R6)
add  R3,R1,R4
sub  R6,R7,R8
```

Suponiendo que se puede escribir un dato en el banco de registros y leer su nuevo valor en el mismo ciclo, calcular el número de ciclos necesarios para ejecutar este código en los siguientes casos:

- No existe la posibilidad de anticipar operandos (NO tiene implementado cortocircuito) ni de reordenar el código.
- Existe anticipación de operandos (tiene implementado cortocircuito), pero NO existe la posibilidad de reordenación de código.
- No hay anticipación de operandos (NO tiene implementado cortocircuito) pero existe la posibilidad de reordenación de código. Reordenar el código para conseguir que el número de ciclos sea mínimo.

4. Sea un DLX segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (tiene implementado cortocircuito).
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- Las unidades funcionales del procesador son:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	sí
FP MUL	1	5	sí
Int ALU	1	1	No

Sabiendo que el siguiente fragmento de código se ejecuta sobre dicho procesador

```
LS    F10,0(R1)
MULS  F4,F0,F10
LS    F12,0(R2)
ADDS  F2,F12,F4
LS    F4 8(R1)
MULS  F12,F4,F12
LS    F12,16(R1)
```

- Representar el diagrama instrucción-tiempo para la primera iteración e indicar los cortocircuitos realizados. Indicar claramente las paradas y sus causas.
- Determinar el CPI

5. Supongamos un computador como el DLX segmentado en cinco etapas con las siguientes características:

- Posee anticipación de operandos (tiene implementado cortocircuito tanto a la etapa EX como a la etapa ID).
- Utiliza saltos retardados con un "delay-slot" de un ciclo.
- Posee una sola memoria cache para el almacenamiento de instrucciones y datos. Por esta razón no resulta posible leer una instrucción y realizar la lectura o escritura de un dato en el mismo ciclo de reloj (no puede coincidir etapa MEM de un load o store con otra instrucción en etapa IF).
- Las escrituras en el banco de registros se hacen en la primera mitad de la fase WB, mientras que las lecturas se hacen en la segunda mitad de la fase WB.

Supongamos que este computador ejecuta el siguiente programa:

```
BUCLE:  LW    R2,X(R6)
        LW    R3,Y(R6)
        SUB   R2,R2,R3
        ADD   R2,R2,R1
        SUB   R6,R6,#4
        SW    R2, Z(R6)
        BNEZ  R6,BUCLE
        ADD   R1,R1,#1
        SUB   R3,R3,R7
```

Se supone que el valor inicial de R6 es 2000.

- a) Construye el diagrama de tiempo correspondiente a la primera iteración del bucle, indicando sobre el diagrama los cortocircuitos que se activan.
- b) Calcula el valor del CPI
- c) Calcular el CPI si cambia la gestión de los saltos. Ahora, después del salto siempre **se busca la instrucción siguiente y en caso de que se produzca el salto se anula la instrucción buscada.**

6. El siguiente fragmento de código se ejecuta en un DLX con segmentación de 7 etapas:

```
LOOP:  LS     F2, 0(R1)
        MULTS F4, F2, F0
        LS     F6, 0(R2)
        ADDS   F6, F4, F6
        SS     0(R2), F6
        ADDI   R1, R1, 8
        SGTI   R3, R1, DONE  (PONE R3 =1 SI R1>=DONE)
        BEQZ   R3, LOOP
        ADDI   R2, R2, 8
```

Suponiendo que:

- Tanto la memoria de datos como la de instrucciones están segmentadas en dos etapas
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo
- Se dispone de lógica de cortocircuito tanto a la etapa EX como a la etapa ID.
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación
- Los riesgos EDE se resuelven mediante inhibición de escritura
- Los saltos se resuelven en la etapa de decodificación
- Se emplea un *branchdelay slot* (salto retardado) de una instrucción
- Una inst. aritmética y un store pueden coexistir en las etapas de MEM y WB
- Las unidades funcionales tienen las siguientes características:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	4	sí
FP MUL	1	5	sí
Int ALU	1	1	No

- Determinar el CPI teniendo en cuenta que $done=0x1000$, $r1=0x0100$ y $R3=0$ al inicio de la ejecución.
- Reordenar/modificar el código para que el CPI sea mínimo.
- Realizar este apartado partiendo del código original. Determinar el CPI suponiendo que **NO hay cortocircuito a la etapa ID.**
- Realizar este apartado partiendo del código original. Determinar el CPI suponiendo que NO hay salto retardado y los saltos se resuelven de la siguiente manera: Después del salto siempre se **busca la instrucción siguiente y en caso de que se produzca el salto se anula la instrucción buscada.**

7. En un procesador segmentado es necesario ejecutar el siguiente código:

```
for ( n = 0; n < 100; n++ ) {  
    for ( k = 0; k < 4; k++ ) {  
        out[n] += filter[k] * input[n+k];  
    }  
}
```

Queriendo conseguir el mayor rendimiento, se decide realizar la siguiente implementación en ensamblador:

```
loop_n : ADDI R2,R0,#0  
loop_k : LS   F3,0(R3) ; lee de memoria 4 bytes (load de un float)jæ  
          LS   F4,0(R4) ; lee de memoria 4 bytes (load de un float)  
          MULS F6,F3,F4 ; mult. en punto flotante (de dos float)  
          LS   F5,0(R5) ; lee de memoria 4 bytes (load de un float)  
          ADDS F5,F6,F5 ; suma en punto flotante (de dos float)  
          SS   0(R5),F5 ; escribe en memoria 4 bytes (store de un float)  
          ADDI R3,R3,#4 ; actualizo puntero a filter  
          ADDI R4,R4,#4 ; actualizo puntero a input  
          ADDI R2,R2,#1  
          BLTI R2,#4, loop_k ; salta a loop_k si R2 < 4  
          ADDI R5,R5,#4 ; avanzamos el puntero sobre out  
          SUBI R3,R3,#16 ; R3 vuelve a apuntar a filter[0]  
          SUBI R4,R4,#12 ; R4 apunta al siguiente input[n]  
          ADDI R1,R1,#1  
          BLTI R1,#100,loop_n ; salta a loop_n si R1 < 100  
B.
```

Se supone que:

- Inicialmente R1=0, R3 apunta a filter[0], R4 apunta a input[0] y R5 apunta a out[0]
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo.
- Se dispone de lógica de cortocircuito tanto a la etapa EX como a la etapa ID.
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante paradas hasta que la instrucción ya lanzada entre en la etapa de memoria.
- Los saltos se resuelven en la etapa de decodificación y NO se implementan saltos retardados (se cancela la siguiente instrucción si el salto es tomado)
- Se dispone de las siguientes unidades funcionales:

UF	Cantidad	Latencia	Segmentación
FP ADDS/ SUBS	1	2	Sí
FP MULS	1	3	Sí
INT ALU	1	1	No

- a) Rellena 25 ciclos del diagrama instrucción-tiempo para el bucle interno con $k=3$ y $n = 0$.
- b) Estima el número de ciclos que tardará en ejecutarse el código completo y determina el CPI. Indica claramente los cálculos que realices para llegar a los resultados.
- c) Queremos implementar saltos retardados de un delay-slot. Haz la función del compilador y recoloca el código para saltos retardados. ¿Se obtiene algún beneficio para este código? Justifica tu respuesta.